

4η Εργαστηριακή Άσκηση

Γρηγόρης Δελατόλας 03119202

Γιώργος Κυριάκος 03119204

1.1

Κώδικας της main:

```
/*
 * Step 1: Print the virtual address space layout of this process.
 */

printf(RED "\nStep 1: Print the virtual address space map of this "
        "process [%d].\n" RESET, mypid);
press_enter();

/*
 * TODO: Write your code here to complete Step 1.
 */

show_maps();

/*
 * Step 2: Use mmap to allocate a buffer of 1 page and print the map
 * again. Store buffer in heap_private_buf.
 */
printf(RED "\nStep 2: Use mmap(2) to allocate a private buffer of "
        "size equal to 1 page and print the VM map again.\n" RESET);
press_enter();
/*
 * TODO: Write your code here to complete Step 2.
 */

heap_private_buf = mmap(NULL, get_page_size(), PROT_READ|PROT_WRITE, MAP_ANONYMOUS|MAP_PRIVATE, -1, 0);

if (heap_private_buf == MAP_FAILED) {
    perror("Could not mmap");
    return 1;
}

/*
char *ptr = addr;
printf("%c \n", * ptr);
*/
show_maps();
show_va_info(heap_private_buf);
```

```

/*
 * Step 3: Find the physical address of the first page of your buffer
 * in main memory. What do you see?
 */
printf(RED "\nStep 3: Find and print the physical address of the "
        "buffer in main memory. What do you see?\n" RESET);
press_enter();
/*
 * TODO: Write your code here to complete Step 3.
 */
//uint64_t

get_physical_address(heap_private_buf);
/*
 * Step 4: Write zeros to the buffer and repeat Step 3.
 */
printf(RED "\nStep 4: Initialize your buffer with zeros and repeat "
        "Step 3. What happened?\n" RESET);
press_enter();
/*
 * TODO: Write your code here to complete Step 4.
 */
int i = 0;
for(i = 0; i < buffer_size; i++)
{
    heap_private_buf[i] = 0;
}
// show_maps();
uint64_t pae = get_physical_address(heap_private_buf);
printf("%x \n", pae);
/*
 * Step 5: Use mmap(2) to map file.txt (memory-mapped files) and print
 * its content. Use file_shared_buf.
 */
printf(RED "\nStep 5: Use mmap(2) to read and print file.txt. Print "
        "the new mapping information that has been created.\n" RESET);
press_enter();
/*
 * TODO: Write your code here to complete Step 5.
 */
fd = open("file.txt", O_RDONLY);
file_shared_buf = mmap(NULL, buffer_size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
get_physical_address(file_shared_buf);

I
show_maps();
show_va_info(file_shared_buf);

```

```

/*
 * Step 6: Use mmap(2) to allocate a shared buffer of 1 page. Use
 * heap_shared_buf.
 */
printf(RED "\nStep 6: Use mmap(2) to allocate a shared buffer of size "
        "equal to 1 page. Initialize the buffer and print the new "
        "mapping information that has been created.\n" RESET);
press_enter();
/*
 * TODO: Write your code here to complete Step 6.
 */

heap_shared_buf = mmap(NULL, get_page_size(), PROT_EXEC|PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);

if (heap_shared_buf == MAP_FAILED) {
    perror("Could not mmap");
    return 1;
}

show_maps();
show_va_info(heap_shared_buf);

```

Κώδικας για child:

```
/*
 * Step 7 - Child
 */
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
/*
 * TODO: Write your code here to complete child's part of Step 7.
 */
show_maps();

/*
 * Step 8 - Child
 */
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
/*
 * TODO: Write your code here to complete child's part of Step 8.
 */

pa = get_physical_address(heap_private_buf);
printf("%x \n", pa);

/*
 * Step 9 - Child
 */
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
/*
 * TODO: Write your code here to complete child's part of Step 9.
 */

int i = 0;
for(i = 0; i < buffer_size; i++){
    heap_private_buf[i] = 0;
}

pa = get_physical_address(heap_private_buf);
printf("%x \n", pa);
```

```

/*
 * Step 10 - Child
 */
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
/*
 * TODO: Write your code here to complete child's part of Step 10.
 */

for(i = 0; i < buffer_size; i++){
    heap_shared_buf[i] = 0;
}

pa = get_physical_address(heap_shared_buf);
printf("%x \n", pa);

/*
 * Step 11 - Child
 */
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
/*
 * TODO: Write your code here to complete child's part of Step 11.
 */

if (mprotect(heap_shared_buf, buffer_size, PROT_READ) != 0) {
    perror("Couldn't mprotect");
    exit(errno);
}

show_va_info(heap_shared_buf);

```

Κώδικας για father:

```
/*
 * Step 7: Print parent's and child's maps. What do you see?
 * Step 7 - Parent
 */
printf(RED "\nStep 7: Print parent's and child's map.\n" RESET);
press_enter();

/*
 * TODO: Write your code here to complete parent's part of Step 7.
 */

show_maps();

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

/*
 * Step 8: Get the physical memory address for heap_private_buf.
 * Step 8 - Parent
 */
printf(RED "\nStep 8: Find the physical address of the private heap "
        "buffer (main) for both the parent and the child.\n" RESET);
//sleep(1);
press_enter();

/*
 * TODO: Write your code here to complete parent's part of Step 8.
 */

pa = get_physical_address(heap_private_buf);
printf("%x \n", pa);

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");
```

```

/*
 * Step 9: Write to heap_private_buf. What happened?
 * Step 9 - Parent
 */
printf(RED "\nStep 9: Write to the private buffer from the child and
          "repeat step 8. What happened?\n" RESET);
press_enter();

/*
 * TODO: Write your code here to complete parent's part of Step 9.
 */
pa = get_physical_address(heap_private_buf);
printf("%x \n", pa);

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

/*
 * Step 10: Get the physical memory address for heap_shared_buf.
 * Step 10 - Parent
 */
printf(RED "\nStep 10: Write to the shared heap buffer (main) from "
        "child and get the physical address for both the parent and "
        "the child. What happened?\n" RESET);
press_enter();

/*
 * TODO: Write your code here to complete parent's part of Step 10.
 */
//wait(NULL);
//sleep(3);
heap_shared_buf[0]=0;
pa = get_physical_address(heap_shared_buf);
printf("%x \n", pa);

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

```

```

* Step 11: Disable writing on the shared buffer for the child
* (hint: mprotect(2)).
* Step 11 - Parent
*/
printf(RED "\nStep 11: Disable writing on the shared buffer for the "
        "child. Verify through the maps for the parent and the "
        "child.\n" RESET);
press_enter();

/*
* TODO: Write your code here to complete parent's part of Step 11.
*/

show_va_info(heap_shared_buf);

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, 0))
    die("waitpid");

```

Output:

Step 1: Print the virtual address space map of this process [17000].

```
Virtual Memory Map of process [17000]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e3000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebeb000-7ffe2ebef000 r--p 00000000 00:00 0 [vvar]
7ffe2ebef000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
-----
```

Step 2: Use mmap(2) to allocate a private buffer of size equal to 1 page and print the VM map again.

```
Virtual Memory Map of process [17000]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e3000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebeb000-7ffe2ebef000 r--p 00000000 00:00 0 [vvar]
7ffe2ebef000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
-----
7fb1e20e2000-7fb1e20e8000 rw-p 00000000 00:00 0
```

Step 3: Find and print the physical address of the buffer in main memory. What do you see?

VA[0x7fb1e20e3000] is not mapped; no physical memory allocated.

Step 4: Initialize your buffer with zeros and repeat Step 3. What happened?

8eala000

Step 5: Use mmap(2) to read and print file.txt. Print the new mapping information that has been created.

VA[0xffffffffffffffff] is not mapped; no physical memory allocated.

```
Virtual Memory Map of process [17000]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e2000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebcb000-7ffe2ebec000 r--p 00000000 00:00 0 [vvar]
7ffe2ebec000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
-----
```

VA is not allocated!

Step 6: Use mmap(2) to allocate a shared buffer of size equal to 1 page. Initialize the buffer and print the new mapping information that has been created.

```
Virtual Memory Map of process [17000]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e1000-7fb1e20e2000 rw-p 00000000 00:00 0
7fb1e20e2000-7fb1e20e3000 rwxs 00000000 00:04 6292387 /dev/zero (deleted)
7fb1e20e3000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebcb000-7ffe2ebec000 r--p 00000000 00:00 0 [vvar]
7ffe2ebec000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
-----
7fb1e20e2000-7fb1e20e3000 rwxs 00000000 00:04 6292387 /dev/zero (deleted)
```

Step 7: Print parent's and child's map.

```
Virtual Memory Map of process [17000]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e1000-7fb1e20e2000 rw-p 00000000 00:00 0
7fb1e20e2000-7fb1e20e3000 rwxp 00000000 00:04 6292387 /dev/zero (deleted)
7fb1e20e3000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebcb000-7ffe2ebec000 r--p 00000000 00:00 0 [vvar]
7ffe2ebec000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
-----
```

```
Virtual Memory Map of process [17001]:
00400000-00402000 r-xp 00000000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
00602000-00603000 rw-p 00002000 00:21 9451652 /home/oslab/oslab74/ex4/1.1/mmap
009ed000-00a0e000 rw-p 00000000 00:00 0 [heap]
7fb1e1b1d000-7fb1e1cbe000 r-xp 00000000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1cbe000-7fb1e1ebe000 ---p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ebe000-7fb1e1ec2000 r--p 001a1000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec2000-7fb1e1ec4000 rw-p 001a5000 08:01 6032227 /lib/x86_64-linux-gnu/libc-2.19.so
7fb1e1ec4000-7fb1e1ec8000 rw-p 00000000 00:00 0
7fb1e1ec8000-7fb1e1ee9000 r-xp 00000000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20db000-7fb1e20de000 rw-p 00000000 00:00 0
7fb1e20e1000-7fb1e20e2000 rw-p 00000000 00:00 0
7fb1e20e2000-7fb1e20e3000 rwxp 00000000 00:04 6292387 /dev/zero (deleted)
7fb1e20e3000-7fb1e20e8000 rw-p 00000000 00:00 0
7fb1e20e8000-7fb1e20e9000 r--p 00020000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20e9000-7fb1e20ea000 rw-p 00021000 08:01 6032224 /lib/x86_64-linux-gnu/ld-2.19.so
7fb1e20ea000-7fb1e20eb000 rw-p 00000000 00:00 0
7ffe2ebc2000-7ffe2ebe3000 rw-p 00000000 00:00 0 [stack]
7ffe2ebcb000-7ffe2ebec000 r--p 00000000 00:00 0 [vvar]
7ffe2ebec000-7ffe2ebf0000 r-xp 00000000 00:00 0 [vdso]
```

Step 8: Find the physical address of the private heap buffer (main) for both the parent and the child.

```
8eala000
8eala000
```

Step 9: Write to the private buffer from the child and repeat step 8. What happened?

```
8eala000
7a71000
```

Step 10: Write to the shared heap buffer (main) from child and get the physical address for both the parent and the child. What happened?

```
1c225000
1c225000
```

Step 11: Disable writing on the shared buffer for the child. Verify through the maps for the parent and the child.

```
7fb1e20e2000-7fb1e20e3000 rwxp 00000000 00:04 6292387 /dev/zero (deleted)
7fb1e20e2000-7fb1e20e3000 r--s 00000000 00:04 6292387 /dev/zero (deleted)
```

1.2.1

```
/*
 * mandel.c
 *
 * A program to draw the Mandelbrot Set on a 256-color xterm.
 *
 */

#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <stdint.h>

/* TODOC header file for m(un)map*/

#include "mandel-lib.h"

#define MANDEL_MAX_ITERATION 100000

sem_t *sems_shared_buff;
int NPROCS;

/*****
 * Compile-time parameters *
 *****/

/*
 * Output at the terminal is is x_chars wide by y_chars long
 */
int y_chars = 50;
int x_chars = 90;

/*
 * The part of the complex plane to be drawn:
 * upper left corner is (xmin, ymax), lower right corner is (xmax, ymin)
 */
double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

/*
 * Every character in the final output is
 * xstep x ystep units wide on the complex plane.
 */
double xstep;

"mandel-fork.c" 259L, 5089C
```

```

double xstep;
double ystep;

/*
 * This function computes a line of output
 * as an array of x_char color values.
 */
void compute_mandel_line(int line, int color_val[])
{
    /*
     * x and y traverse the complex plane.
     */

    double x, y;

    int n;
    int val;

    y = ymax - ystep * line;
    for (x = xmin, n = 0; n < x_chars; x+= xstep, n++) {

        /* Compute the point's color value */
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);
        //printf("val before if is %d \n", val);
        if (val > 255)
            val = 255;

        /* And store it in the color_val[] array */
        val = xterm_color(val);
        //printf("color val is %d \n", val);
        color_val[n] = val;
    }
    //printf("end of compute_mandel_line");
}

/*
 * This function outputs an array of x_char color values
 * to a 256-color xterm.
 */
int main(int argc, char **argv)
{
    int i;
    int color_val[256];
    compute_mandel_line(0, color_val);
    for (i = 0; i < 256; i++)
        printf("%c", color_val[i]);
    return 0;
}

```

```
void output_mandel_line(int fd, int color_val[])
{
    int i;

    char point = '@';
    char newline = '\n';

    for (i = 0; i < x_chars; i++) {
        /* Set the current color, then output the point */
        set_xterm_color(fd, color_val[i]);
        if (write(fd, &point, 1) != 1) {
            perror("compute_and_output_mandel_line: write point");
            exit(1);
        }
    }

    /* Now that the line is done, output a newline character */
    if (write(fd, &newline, 1) != 1) {
        perror("compute_and_output_mandel_line: write newline");
        exit(1);
    }
}
```

```
void compute_and_output_mandel_line(int fd, int line)
{
    int color_val[x_chars];

    int j;
    for(j = line; j < y_chars; j = j + NPROCS){

        compute_mandel_line(j, color_val);

        int mod = j % NPROCS;

        if(mod == 0){

            sem_wait(&sems_shared_buff[NPROCS-1]);

            output_mandel_line(1, color_val);

            sem_post(&sems_shared_buff[0]);

        }
        else{

            sem_wait(&sems_shared_buff[mod-1]);

            output_mandel_line(1, color_val);

            sem_post(&sems_shared_buff[mod]);

        }

    }

}
```

```

void *create_shared_memory_area(unsigned int numbytes) // numbytes = number_of_semaphores * bytes_per_s
{
    int pages;
    sem_t *addr;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    /*
     * Determine the number of pages needed, round up the requested number of
     * pages
     */
    pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;

    /* Create a shared, anonymous mapping for this number of pages */
    /* TODO:
       addr = mmap(...)
    */

    addr = mmap(NULL, pages * sysconf(_SC_PAGE_SIZE), PROT_WRITE|PROT_READ, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
    if (addr == MAP_FAILED) {
        perror("Could not mmap");
        return 1;
    }

    return addr;
}

void destroy_shared_memory_area(void *addr, unsigned int numbytes) {
    int pages;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    /*
     * Determine the number of pages needed, round up the requested number of
     * pages
     */
    pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;

    if (munmap(addr, pages * sysconf(_SC_PAGE_SIZE)) == -1) {
        perror("destroy_shared_memory_area: munmap failed");
        exit(1);
    }
}

```

```

int main(int argc, char *argv[])
{
    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    int i;

    NPROCS = atoi(argv[1]);

    unsigned int numbytes = NPROCS * sizeof(sem_t);

    sems_shared_buff = create_shared_memory_area(numbytes);

    // printf("before sems init \n");

    for(i = 0; i < NPROCS-1; i++){

        sem_init(&sems_shared_buff[i], 1, 0);
    }

    sem_init(&sems_shared_buff[NPROCS-1], 1, 1);

    //printf("%c \n", heap_shared_buff[1]);

    // printf("after sems init \n");

    pid_t pid;

    // create n child processes

    for(i = 0; i < NPROCS; i++){

        //printf("before fork \n");
        pid = fork();

        if(pid < 0) {
            perror("error in fork");
            exit(1);
        }

        else if(pid == 0){

            //printf("%d process before compute_and_output_mandel_line \n", i);
            compute_and_output_mandel_line(1, i);
            exit(1);
        }

        // printf("end of for loop no %d \n", i);
    }
}

```

```
for(i = 0; i < NPROCS; i++){  
    wait(NULL);;  
}  
  
//printf("Before resetting the color \n");  
int line;  
  
reset_xterm_color(1);  
  
destroy_shared_memory_area(sems_shared_buff, numbytes);  
  
return 0;
```

Με χρήση της εντολής time παρατηρήσαμε ότι για μικρό N τα threads και οι διεργασίες έχουν σχεδόν ίδια απόδοση. Ωστόσο, όσο το N αυξάνει τα threads γίνονται λίγο ταχύτερα.

Στην υλοποίηση με τα νήματα η μνήμη είναι έτσι και αλλιώς διαμοιραζόμενη μεταξύ των νημάτων. Αντιθέτως, στην υλοποίηση με τις διεργασίες χρειάζεται εμείς να φτιάξουμε τη διαμοιραζόμενη μνήμη, γεγονός που δυσχεραίνει την επίδοση.

1.2.2

```

void compute_mandel_line(int line, int color_val[])
{

    //printf("In compute mandel line for line: %d \n", line);
    /*
     * x and y traverse the complex plane.
     */
    double x, y;

    int n;
    int val;

    /* Find out the y value corresponding to this line */
    y = ymax - ystep * line;

    /* and iterate for all points on this line */
    for (x = xmin, n = 0; n < x_chars; x+= xstep, n++) {

        /* Compute the point's color value */
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);
        if (val > 255)
            val = 255;

        /* And store it in the color_val[] array */
        val = xterm_color(val);
        //printf("Before color val insert... \n");
        //if(line == 0 || line == 30){
        //    printf("Row is: %d \n", line);
        //    printf("Column is: %d \n", n);
        //    printf("Val is: %d \n \n", val);
        //}
        color_val[line*x_chars + n] = val;
        //printf("after color val insert... \n");
    }
    //output_mandel_line(1, color_val, line);

    //printf("Before next line... \n");

    if(line + NPROCS < y_chars){

        compute_mandel_line(line+NPROCS, color_val);

    }
}

```

```

void output_mandel_line(int fd, int color_val[], int line)
{
    int i;

    char point = '@';
    char newline = '\n';

    for (i = 0; i < x_chars; i++) {
        /* Set the current color, then output the point */
        set_xterm_color(fd, color_val[line*x_chars + i]);
        if (write(fd, &point, 1) != 1) {
            perror("compute_and_output_mandel_line: write point");
            exit(1);
        }
    }

    /* Now that the line is done, output a newline character */
    if (write(fd, &newline, 1) != 1) {
        perror("compute_and_output_mandel_line: write newline");
        exit(1);
    }
}

-----

/*****
 * Compile-time parameters *
 *****/

int *shared_buffer;
int NPROCS;

/*
 * Output at the terminal is is x_chars wide by y_chars long
 */
int y_chars = 50;
int x_chars = 90;

/*
 * The part of the complex plane to be drawn:
 * upper left corner is (xmin, ymax), lower right corner is (xmax, ymin)
 */
double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

/*
 * Every character in the final output is
 * xstep x ystep units wide on the complex plane.
 */
double xstep;
double ystep;

```

```

void *create_shared_memory_area(unsigned int numbytes)
{
    int pages;
    void *addr;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    /*
     * Determine the number of pages needed, round up the requested number of
     * pages
     */
    pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;

    /* Create a shared, anonymous mapping for this number of pages */
    /* TODO: addr = mmap(...) */
    /*
    i addr = mmap(NULL, pages * sysconf(_SC_PAGE_SIZE), PROT_READ|PROT_WRITE, MAP_ANON|MAP_SHARED, -1, 0);
    */

    return addr;
}

void destroy_shared_memory_area(void *addr, unsigned int numbytes) {
    int pages;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    /*
     * Determine the number of pages needed, round up the requested number of
     * pages
     */
    pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;

    if (munmap(addr, pages * sysconf(_SC_PAGE_SIZE)) == -1) {
        perror("destroy_shared_memory_area: munmap failed");
        exit(1);
    }
}

```

```

int main(int argc, char *argv[])
{
    NPROCS = atoi(argv[1]);
    int line;

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    /*
I    * draw the Mandelbrot Set, one line at a time.
    * Output is sent to file descriptor '1', i.e., standard output.
    */

    int i;

    int numbytes = sizeof(int) * y_chars * x_chars;
    shared_buffer = create_shared_memory_area(numbytes);

    pid_t pid;

    /*
    for (line = 0; line < y_chars; line++) {
        compute_and_output_mandel_line(1, line);
    }
    */
    //printf("Before fork ... \n");

    for(i = 0; i < NPROCS; i++){

        //printf("before fork \n");
        p = fork();

        if(pid < 0) {
            perror("error in fork");
            exit(1);
        }

        else if(pid == 0){

            //printf("%d process before compute_mandel_line \n", i);
            compute_mandel_line(i, shared_buffer);
            exit(1);
        }
    }
}

```

```
for(i = 0; i < NPROCS; i++){  
    pid = wait(NULL);  
}  
  
for(i = 0; i < y_chars; i++){  
    output_mandel_line(1, shared_buffer, i);  
}  
  
reset_xterm_color(1);  
return 0;
```

Ο συγχρονισμός επιτυγχάνεται με το wait του πατέρα για το παιδί του. Στην ουσία κάθε παιδί υπολογίζει τις γραμμές στις οποίες έχει αντιστοιχηθεί χωρίς να μας ενδιαφέρει η γραμμική σειρά υπολογισμού. Έπειτα, ο πατέρας κάνει wait τα παιδιά του και στην συνέχεια τυπώνει τις γραμμές σειριακά.

Στην περίπτωση που είχαμε έναν buffer με διαστάσεις NPROCS x x_chars η απόδοση θα μειωνόταν καθώς θα έπρεπε κάθε παιδί να υπολογίζει την γραμμή του, να τυπώνεται το κάθε set γραμμών με τη σωστή σειρά, και έπειτα να υπολογίζονται οι επόμενες γραμμές. Άρα, θα έπρεπε να συγχρονίζεται και ο υπολογισμός και η εκτύπωση.